



I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 4: Aprendizaje No-Supervisado

Martin Bianchi

28 de mayo de 2025

Ingeniería en Inteligencia Artificial

1. Clustering de datos

Resumen

En este trabajo se abordó el problema de agrupamiento de datos aplicando tres algoritmos distintos: K-Means, Gaussian Mixture Models (GMM) y DBSCAN. Para ello, se utilizó un conjunto de datos bidimensional (`clustering.csv`) que fue preprocesado mediante estandarización. K-Means determinó el número óptimo de clusters ($K = 14$) mediante el método del codo, mientras que GMM capturó estructuras elípticas con mayor precisión. DBSCAN permitió identificar agrupaciones densas y puntos de ruido. Los resultados mostraron que GMM fue el modelo más efectivo para representar la estructura subyacente de los datos, superando a K-Means en casos de clusters no esféricos y a DBSCAN en regiones densas cercanas. Las métricas de evaluación incluyeron inercia, log-verosimilitud y silhouette score, destacándose el GMM en términos de ajuste y flexibilidad.

1.1. Introducción

En este trabajo se aborda el problema de agrupamiento de datos (clustering) utilizando el conjunto `clustering.csv`, que contiene dos variables numéricas representadas en un espacio bidimensional. El objetivo principal es identificar grupos naturales dentro de los datos y analizar las características de cada cluster resultante.

La entrada de los algoritmos implementados consiste en las coordenadas (A, B) de cada punto en el plano, mientras que la salida corresponde a la asignación de cada punto a un cluster específico o, en algunos casos, su clasificación como ruido. Los métodos utilizados incluyen el algoritmo K-means, el modelo de mezclas gaussianas (GMM) y DBSCAN. Cada uno de estos algoritmos presenta fortalezas y limitaciones específicas que permiten explorar diferentes estructuras en los datos.

El análisis se realiza en varias etapas. En primer lugar, se aplica el método de K-means para determinar la cantidad óptima de clusters mediante el criterio de "ganancias decrecientes", evaluando cómo la suma de las distancias a los centroides varía con el número de clusters. Luego, se utiliza el modelo GMM para identificar agrupamientos elípticos, inicializando la optimización con los resultados de K-means. Finalmente, se emplea DBSCAN, que clasifica puntos densamente conectados y detecta puntos aislados como ruido. Este último método permite analizar agrupaciones de formas arbitrarias sin requerir la especificación previa del número de clusters.

El propósito de este trabajo es comparar el desempeño de estos tres enfoques sobre un mismo conjunto de datos, identificando las fortalezas y limitaciones de cada método. Los resultados incluyen visualizaciones de los clusters obtenidos, la identificación de parámetros óptimos y un análisis crítico sobre la capacidad de los algoritmos para capturar la estructura subyacente de los datos. Esto permite no solo comprender el comportamiento de los algoritmos en este caso particular, sino también extrapolar conclusiones aplicables a problemas similares.

1.2. Métodos

Se trabajó con el conjunto de datos `clustering.csv`. En la etapa de preprocesamiento se realizó una exploración inicial del dataset para detectar valores atípicos, filas duplicadas y verificar la escala de las variables. Se observó que las columnas presentaban valores en un rango amplio (0 a 10^6), lo que motivó la aplicación de una transformación de estandarización. Luego

de dicha transformación, ambas columnas quedaron centradas alrededor de 0 y contenidas entre -2 y 2, sin distorsionar la forma de las distribuciones originales.

- **K-Means:** Se implementó el algoritmo utilizando la inicialización `KMeans++`, con 15 repeticiones y selección del mejor resultado según la mínima inercia. El número óptimo de clusters fue **K=14**, determinado mediante el método del codo. Cada punto x_i fue asignado al cluster más cercano en distancia euclídea, y se graficaron los datos con colores por cluster y centroides marcados.
- **Gaussian Mixture Model (GMM):** Se implementó como alternativa más flexible, inicializando con la solución previa de K-Means. Se usó la log-verosimilitud para elegir el número de componentes, y el modelo con **K=14** permitió representar formas elípticas, capturando mejor la estructura de algunos grupos que K-Means.
- **DBSCAN:** Explorando combinaciones de ε (radio de vecindad) y `min_samples`, se seleccionó $\varepsilon=0.2$ y `min_samples=50`, maximizando la métrica `silhouette score`. Este enfoque permitió identificar puntos ruidosos y clusters densos, aunque mostró limitaciones para formas elípticas.

Conjunto de datos y validación

El dataset fue dividido utilizando la totalidad de los datos sin separación explícita de conjuntos de entrenamiento/validación/test, dado que el objetivo principal fue el análisis exploratorio no supervisado. No se utilizó validación cruzada formal, pero se aplicó exploración de hiperparámetros en cada modelo.

Métricas utilizadas

Las métricas de evaluación incluyeron:

- **Inercia total (L):** suma de las distancias cuadradas de cada punto a su centroide. Se usa en K-Means para evaluar compacidad.
- **Log-verosimilitud:** usada en GMM para medir cuán probable es que los datos provengan del modelo aprendido. Mayor valor implica mejor ajuste.
- **Silhouette manual:** mide qué tan similares son los puntos dentro del mismo cluster en comparación con los de otros clusters. Varía entre -1 y 1; valores cercanos a 1 indican mejor separación.

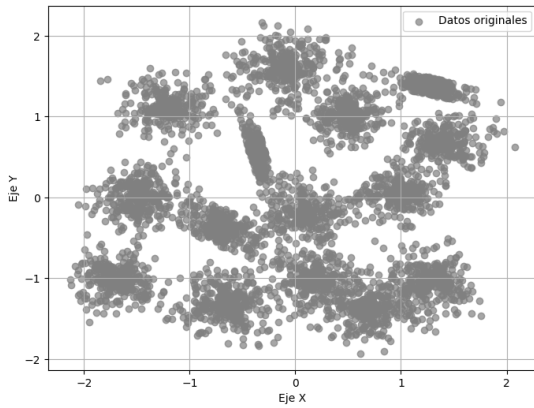
Estas métricas permitieron comparar la calidad de segmentación de los distintos métodos y seleccionar las configuraciones óptimas.

1.3. Análisis

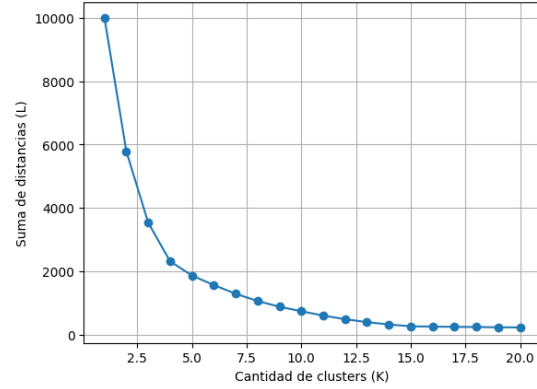
Para explorar la estructura que subyace en los datos, proyectamos las variables A y B en el plano, como se muestra en la Figura 1a. En esta visualización se distinguen aproximadamente 13 agrupaciones principales, aunque algunas regiones vecinas se solapan o presentan diferencias de densidad. Además, no todas las formas son claramente compactas ni bien diferenciadas:

varias agrupaciones tienen contornos más bien elípticos o difusos, lo que dificulta identificar límites precisos y puede afectar el rendimiento de algoritmos más rígidos.

Para tener una idea más clara del número adecuado de clusters, aplicamos el algoritmo **K-Means** y evaluamos la inercia total L en función de K , usando el método de **ganancias decrecientes**, como se muestra en la Figura 1b. Para que los resultados fueran consistentes, usamos la inicialización **KMeans++** y repetimos el procedimiento 15 veces, quedándonos con la corrida que dio menor inercia. Así, obtuvimos una curva L vs. K suave y decreciente, donde la mejora se vuelve marginal entre $K = 13$ y $K = 15$. Esto sugiere que **$K=14$** es una buena elección, y coincide con lo que se observa en la proyección inicial.



(a) Distribución de los datos originales



(b) Ganancias decrecientes de inercia L vs número de clusters K

Figura 1: Visualización inicial de la distribución de datos (a) y curva de ganancias decrecientes para decidir K (b).

Con $K = 14$, los clusters obtenidos con **K-Means** se generan con velocidad y están bien definidos a simple vista, como se observa en la Figura 2a. Sin embargo, este algoritmo agrupa los puntos únicamente según su distancia euclídea a los centroides, lo que lo limita a detectar regiones aproximadamente esféricas. Como resultado, no puede representar correctamente clusters con formas alargadas u orientaciones específicas.

Esto se evidencia en el cluster 3 (anaranjado) de la Figura 2a, donde K-Means agrupa los puntos que forman una estructura elíptica coherente con otros puntos cercanos que, en realidad, pertenecen a una región distinta. Esta mezcla distorsiona la verdadera estructura del agrupamiento.

Para abordar esta limitación, utilizamos un **Gaussian Mixture Model (GMM)** con covarianza completa. A diferencia de K-Means, GMM modela cada cluster como una distribución gaussiana que puede tener forma, tamaño y orientación propios, gracias al uso de matrices de covarianza completas. Esto le permite adaptarse a estructuras elípticas y mejorar la separación entre grupos. Inicializamos el modelo con los centroides obtenidos por K-Means y empleamos nuevamente el criterio de ganancia decreciente, esta vez sobre la log-verosimilitud, que también indica **$K=14$** . Como muestra la Figura 2b, GMM logra una segmentación más precisa: en el caso del cluster naranja 3, por ejemplo, separa correctamente los puntos discordantes que K-Means había incluido erróneamente.

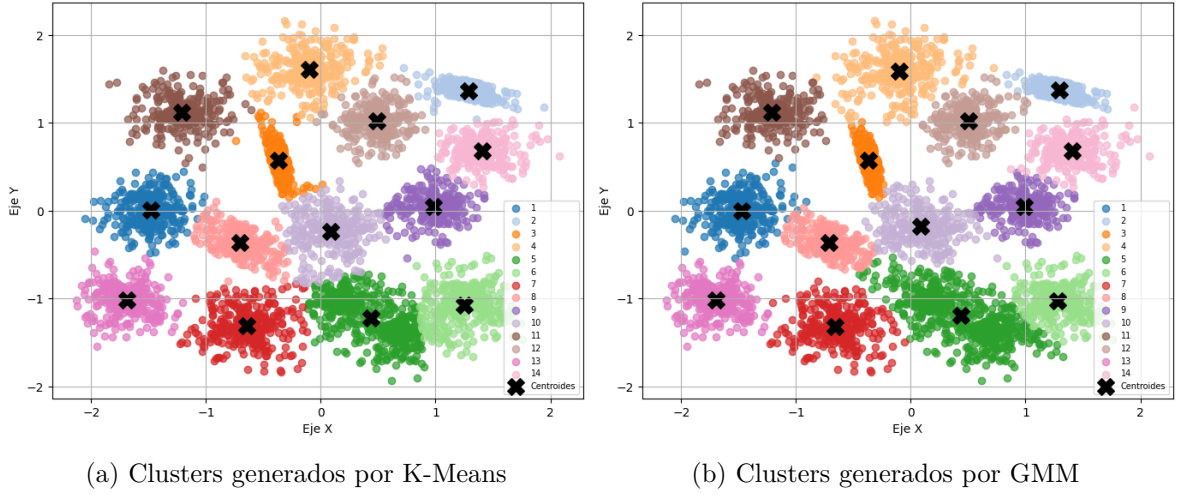


Figura 2: Comparación de los resultados obtenidos con K-Means (a) y GMM (b) para $K = 14$.

No obstante, tanto K-Means como GMM asignan todos los puntos a algún cluster sin poder distinguir explícitamente el ruido, lo que motiva el uso de métodos basados en densidad como DBSCAN.

Finalmente, aplicamos **DBSCAN**, que no asume ninguna forma predefinida de cluster sino que detecta agrupaciones según la densidad y etiqueta como ruido los puntos aislados. Para seleccionar los parámetros, exploramos diferentes valores de ε y `min_samples`, evaluando simultáneamente la **silhouette score** con un umbral de porcentaje de ruido (Figura 3a). El mejor compromiso se alcanzó con $\varepsilon=0.2$ y `min_samples=50`, manteniendo un ruido menor al 5%.

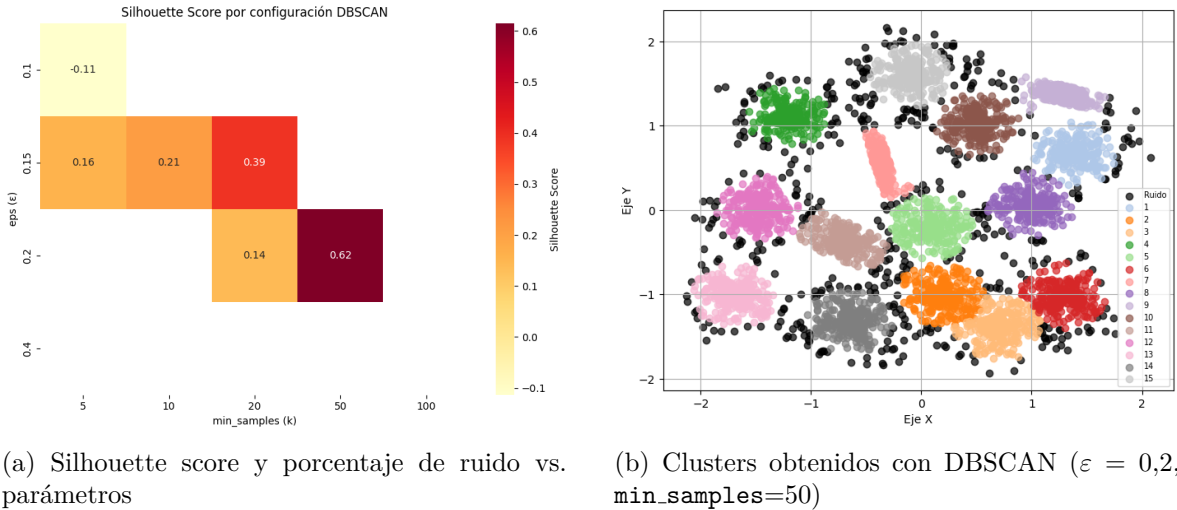


Figura 3: Selección de parámetros y resultado final de DBSCAN; los puntos marcados en gris son ruido.

DBSCAN aprovecha la conectividad por densidad para adaptarse a clusters de formas arbitrarias sin requerir un valor de K . A diferencia de K-Means y GMM, identifica explícitamente

puntos como ruido, evitando asignaciones forzadas en regiones ambiguas. En la Figura 3b se observa que la mayoría de las estructuras se representan con mayor fidelidad, y los puntos no claramente asociados a una forma quedan correctamente etiquetados como ruido. No obstante, en el cluster elíptico persiste el problema de absorción de puntos cercanos pertenecientes a otras regiones, lo que distorsiona su estructura, aunque esta se contiene en su totalidad. Esta respuesta es coherente con la naturaleza del algoritmo, que prioriza la densidad sobre la forma precisa.

En resumen, el modelo que mejor representó la estructura de los datos fue el **GMM**, gracias a su capacidad para adaptarse a formas elípticas y capturar orientaciones específicas, lo que le permitió segmentar agrupaciones que K-Means no pudo distinguir correctamente. **DBSCAN**, si bien ofreció ventajas notables al identificar puntos de ruido y adaptarse a formas no esféricas, fue sensible a la elección de parámetros y no logró separar completamente algunas regiones densas cercanas, como ocurrió en el cluster elíptico. En contraste, **K-Means** resultó ser el método menos adecuado para este conjunto de datos, ya que su supuesto de clusters esféricos condujo a mezclas incorrectas en estructuras alargadas. Estos resultados muestran que los modelos más flexibles, como GMM, tienden a ofrecer una segmentación más fiel cuando las agrupaciones no son perfectamente separables ni uniformes.

2. Reducción de dimensionalidad

Resumen

En este trabajo se abordó el problema de reducción de dimensionalidad y reconstrucción de imágenes aplicando dos enfoques distintos: Análisis de Componentes Principales (PCA) y Autoencoder Variacional (VAE). Se utilizó el conjunto de datos `MNIST_dataset.csv`, compuesto por imágenes de dígitos manuscritos representadas como vectores de 784 dimensiones. PCA permitió proyectar los datos en subespacios lineales de menor dimensión, mientras que el VAE aprendió representaciones no lineales mediante una red neuronal entrenada con regularización probabilística. Ambos métodos fueron evaluados comparando la calidad de las imágenes reconstruidas a través del error cuadrático medio (MSE). Los resultados mostraron que el VAE logró reconstrucciones más precisas y realistas, especialmente con pocas dimensiones latentes, superando a PCA en capacidad de representación y compresión.

2.1. Introducción

En este trabajo se aborda el problema de reducción de dimensionalidad utilizando el conjunto de datos `MNIST_dataset.csv`, que contiene imágenes de dígitos manuscritos del 0 al 9 representadas en forma tabular. Cada imagen es originalmente una grilla de 28×28 píxeles en escala de grises, codificada como un vector de 784 valores que indican la intensidad de gris de cada píxel.

La entrada del algoritmo consiste en estos vectores de 784 dimensiones, mientras que la salida es una versión reconstruida de cada imagen, a partir de una representación comprimida en un espacio de menor dimensión. El objetivo principal es analizar qué tan bien se puede preservar la información visual relevante de las imágenes originales al aplicar técnicas de compresión, evaluando la calidad de la reconstrucción mediante el error cuadrático medio (MSE).

Para ello, se implementó el algoritmo de **Análisis de Componentes Principales (PCA)**, una técnica lineal que permite proyectar los datos en un subespacio de menor dimensión maximizando la varianza retenida. Posteriormente, se entrenó un **autoencoder variacional (VAE)** utilizando redes neuronales, el cual permite aprender codificaciones no lineales más expresivas del conjunto de datos.

El análisis incluye una comparación entre ambas técnicas en términos de precisión de reconstrucción, visualización de las imágenes reconstruidas, y evaluación del impacto que tiene la cantidad de dimensiones utilizadas sobre la calidad de los resultados. Este enfoque permite comprender no solo las limitaciones del PCA frente a modelos más complejos como el VAE, sino también explorar cómo diferentes estrategias de compresión afectan la fidelidad visual en conjuntos de datos de alta dimensionalidad.

2.2. Métodos

Se trabajó con el conjunto de datos `MNIST_dataset.csv`, compuesto por imágenes de dígitos manuscritos (0–9) representadas como vectores de 784 dimensiones (correspondientes a imágenes de 28×28 píxeles).

- **Principal Component Analysis (PCA):** Antes de aplicar PCA, se centraron los datos restando la media de cada característica. Esta centración es necesaria para garantizar que las componentes principales reflejen direcciones de máxima varianza en torno al origen. No se realizó normalización, ya que PCA requiere únicamente que los datos estén centrados.

El método encuentra una base ortogonal de vectores (componentes principales) que maximizan la varianza de los datos proyectados. Para un conjunto de datos centrado $X \in R^{n \times d}$, se calcula la matriz de covarianza $\Sigma = \frac{1}{n} X^\top X$. Luego, se realiza una descomposición espectral de Σ para obtener sus autovectores v_i y autovalores λ_i , ordenados según λ_i de mayor a menor.

La proyección de los datos en un subespacio de dimensión K se obtiene como $Z = XV_K$, donde $V_K \in R^{d \times K}$ contiene los K autovectores principales como columnas. La reconstrucción aproximada de los datos originales se realiza mediante $X_{\text{recon}} = ZV_K^\top = XV_KV_K^\top$.

- **Autoencoder Variacional (VAE):** Es un tipo de red neuronal generativa con arquitectura encoder-decoder simétrica. El encoder mapea la entrada x a una distribución latente $q_\phi(z|x) \sim \mathcal{N}(\mu(x), \sigma^2(x)I)$, y el decoder reconstruye la imagen desde una muestra z de esa distribución. A diferencia de un autoencoder clásico, el VAE impone una estructura probabilística al espacio latente, permitiendo generar nuevas muestras.

Para entrenarlo, los datos se dividieron en **80 % entrenamiento** y **20 % validación**, y se normalizaron al rango $[0, 1]$ dividiendo por 255, ya que la red utiliza activaciones ReLU sensibles a la escala.

La función de pérdida combina un término de reconstrucción con una penalización KL que acerca $q_\phi(z|x)$ a una gaussiana estándar $p(z)$:

$$\mathcal{L} = E_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) \parallel p(z))$$

Se aplica reparametrización $z = \mu + \sigma \cdot \epsilon$, con $\epsilon \sim \mathcal{N}(0, I)$, para permitir el entrenamiento por backpropagation.

Hiperparámetros

Para el VAE, que utiliza una arquitectura simétrica entre encoder y decoder, se probaron distintas combinaciones de hiperparámetros, entre ellos:

- Dimensión del espacio latente: $K \in \{3, 30, 120, 210\}$
- Capas ocultas: [512, 256], [256, 128], [256], [512]
- Tamaño de batch: 64, 128
- Épocas: 20
- Optimización: Adam
- Tasa de aprendizaje: 0.001, 0.01
- Regularización L2: 10^{-5} , 10^{-4} , 10^{-3}

La mejor combinación encontrada fue: capas ocultas [256], dimensión latente $K = 210$, batch size 128, tasa de aprendizaje 0.001 y regularización L2 de 10^{-4} . Se utilizó la misma dimensión latente que en PCA para permitir una comparación directa.

Métricas de evaluación

Se utilizó el **error cuadrático medio (MSE)** como métrica de reconstrucción para ambos métodos, definida como:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \|x_i - \hat{x}_i\|^2$$

donde x_i es la imagen original y $\hat{x}_i$ su reconstrucción. También se empleó inspección visual cualitativa de las imágenes reconstruidas como apoyo.

2.3. Análisis

Comenzamos aplicando **Análisis de Componentes Principales (PCA)** al dataset completo. Dado que cada imagen está representada como un vector de 784 componentes (28x28 píxeles), el objetivo fue determinar cuántos componentes principales son necesarios para mantener una reconstrucción razonablemente fiel. Para esto, calculamos el *error cuadrático medio* (MSE) de reconstrucción en función del número de componentes utilizados, como se muestra en la Figura 4.

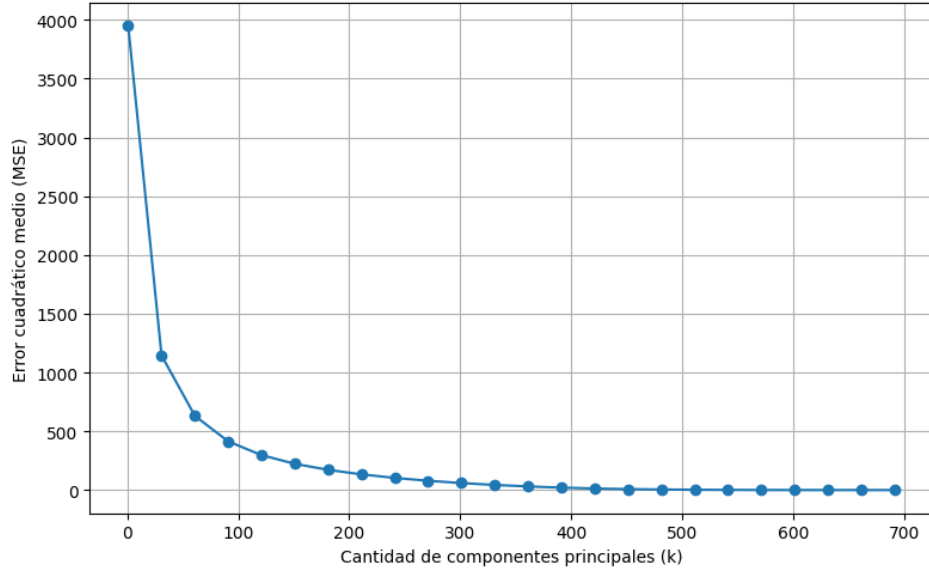


Figura 4: Error cuadrático medio de reconstrucción (MSE) en función del número de componentes principales utilizados en PCA.

La curva muestra una fuerte disminución inicial del error cuadrático medio (MSE) al incrementar el número de componentes principales, lo que indica que gran parte de la varianza del conjunto está concentrada en las primeras direcciones principales. A medida que se aumenta la cantidad de componentes, la mejora en la reconstrucción se mantiene pero con retornos cada vez menores. En particular, entre $k = 1$ y $k = 50$ se observa una caída abrupta del MSE, mientras que más allá de $k = 200$ la pendiente se aplana notablemente y las reducciones en el error se vuelven cada vez más marginales. En este contexto, seleccionamos **$k=210$** con un **MSE de 134** como un punto de equilibrio razonable entre eliminar suficientes dimensiones para lograr compresión y conservar la mayor parte de la información sin incurrir en una pérdida significativa.

Con esta configuración, reconstruimos visualmente las primeras 10 imágenes del conjunto, contrastando los originales con sus reconstrucciones (Figura 5). Se observa que, aunque las imágenes reconstruidas pierden nitidez —con bordes más difusos y un fondo menos oscuro que en las originales—, la estructura general de los dígitos se mantiene claramente reconocible. Esto refleja el hecho de que, al conservar solo las componentes principales de mayor varianza, se retiene la información más relevante para representar la forma de los números, mientras que se descarta parte del contenido asociado a detalles finos o posibles fluctuaciones de bajo nivel. Así, el resultado combina una pérdida de precisión visual con la preservación de las características esenciales, en un compromiso entre compresión y fidelidad.

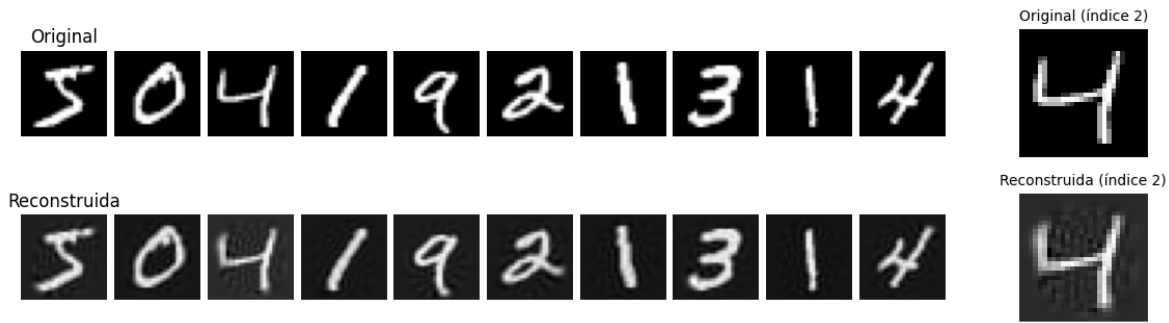


Figura 5: Reconstrucción de imágenes mediante PCA usando $K = 210$ componentes.

A continuación, implementamos un **Autoencoder Variacional (VAE)** y lo entrenamos utilizando la misma cantidad de información latente que en el PCA: un espacio de dimensión **210**. A diferencia de PCA, que se basa exclusivamente en combinaciones lineales de los píxeles, el VAE aprende una codificación no lineal adaptada a la distribución de los datos, lo que lo habilita a capturar estructuras más complejas.

Exploramos diferentes configuraciones arquitectónicas, ajustando hiperparámetros como *tasa de aprendizaje*, *batch size*, *dimensión oculta* y *regularización L2*. La mejor variante encontrada, con una única capa oculta de 256 neuronas y regularización moderada, alcanzó un **MSE de validación de apenas 0.015**, contrastando fuertemente con el valor obtenido por PCA (**134**) para el mismo número de dimensiones.

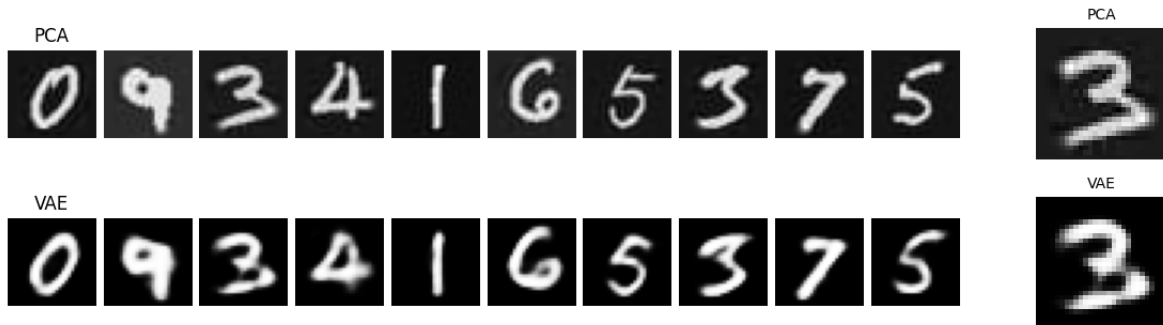


Figura 6: Comparación visual de reconstrucciones PCA vs VAE

Para comparar visualmente los resultados, generamos reconstrucciones sobre el mismo conjunto de 10 imágenes para ambas técnicas (Figura 6).

La diferencia es clara: las reconstrucciones del VAE presentan **mayor nitidez**, **bordes más definidos** y un **fondo más limpio**, con negros puros que mejoran el contraste y realzan la forma de los dígitos. En cambio, las imágenes reconstruidas por PCA, si bien conservan adecuadamente la **estructura general** de los números, exhiben un **fondo degradado** y **contornos difusos**, lo que da lugar a una apariencia más “borrosa” o granulada, especialmente en dígitos como el “3”. Esto sugiere que PCA tiende a conservar la estructura media de los datos, pero no logra reconstruir los detalles finos con la misma precisión que un modelo generativo como el VAE.

Estas diferencias reflejan las limitaciones del carácter puramente lineal de PCA, que no

puede capturar bien las variaciones no lineales presentes en los datos. El VAE, en cambio, aprovecha su capacidad de aprendizaje no lineal para generar representaciones más fieles y visualmente más limpias, adaptadas a la geometría subyacente del conjunto de entrenamiento. A diferencia de PCA, el VAE no memoriza ni copia directamente la entrada: en su lugar, aprende una *distribución latente* desde la cual luego genera nuevas muestras. Por eso, las imágenes reconstruidas por el VAE no son idénticas al original, pero sí *altamente plausibles* y, en muchos casos, más limpias, ya que puede eliminar ruido o variaciones irrelevantes. Este efecto se evidencia en errores de reconstrucción muy bajos, como un MSE de 0.002 en validación, que indican una compresión eficaz sin pérdida visual significativa. Así, el VAE logra una reconstrucción casi perfecta en apariencia, sin necesidad de replicar píxel por píxel la imagen original.

Si bien con 210 dimensiones ya se observa una clara ventaja del VAE, realizamos un análisis adicional para evaluar su desempeño al reducir la **dimensión del espacio latente**. Aun con valores extremadamente bajos, como **latent_dim = 3**, el VAE logró reconstrucciones razonables y un **MSE de aproximadamente 0.04**, ya inferior al obtenido por PCA con 210 componentes. Al incrementar la dimensión a 30, el error disminuye abruptamente hasta alrededor de **0.015**, y más allá de ese punto las mejoras se vuelven marginales. Esto se observa con claridad en la Figura 7, donde se visualiza la relación entre el MSE de validación y la dimensión del espacio latente.

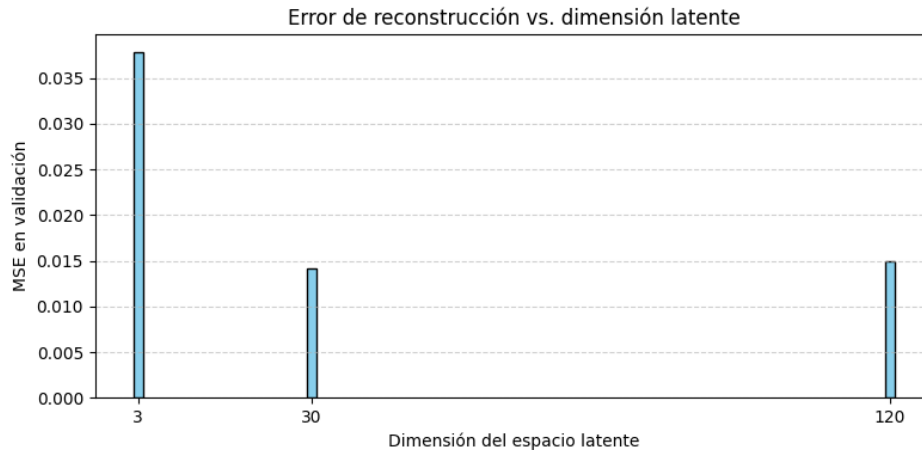


Figura 7: Error de reconstrucción en validación en función de la dimensión del espacio latente. Se observa una fuerte caída inicial en el MSE, con mejoras marginales a partir de 30 dimensiones.

En definitiva, el análisis evidencia que, si bien el PCA permite una compresión significativa preservando la estructura general de las imágenes, su enfoque lineal impone un límite claro en la calidad de reconstrucción. El VAE, al incorporar una codificación no lineal y una representación probabilística, no solo supera ampliamente al PCA en términos de MSE, sino que también ofrece resultados visualmente superiores incluso con espacios latentes mucho más reducidos. Esta capacidad de mantener reconstrucciones nítidas y coherentes con muy poca información latente destaca al VAE como una herramienta más potente y versátil para compresión y modelado generativo en contextos donde la fidelidad visual y la captura de

estructuras complejas son prioritarias.

Apéndice: Enlace al repositorio del proyecto

En el siguiente enlace de Google Drive se encuentran todos los archivos relacionados con el proyecto, incluyendo el notebook explicativo, códigos de los modelos, datos utilizados, un archivo README, los requisitos (`requirements.txt`) y otros recursos necesarios: <https://drive.google.com/drive/folders/1pYLH4ViExUigSSCTPEycBg0eE10J2QrD?usp=sharing>